

Computer Science & IT

Data Structure & programming



Linked List

Lecture No. 03



By- Abhishek Sir

Recap of Previous Lecture



Topic

Single linked List

Topic

Adding a node beginning

Topic

Adding a Node at the end

Topic

Add a node x after y

Topic

Search an element in list

Topics to be Covered



Topic

More operation on single linked List

Topic

Doubly Linked List

Topic

Topic

Topic



Topic Question

If the list contain the value $1 \rightarrow 2 \rightarrow 3$ then what is the Output following function.

```
void foo (Node *ptr) {  
    if (ptr) {  
        printf ("%d", ptr ->data);  
        fun (ptr -> link);  
        fun (ptr -> link);  
        fun (ptr->link);  
        printf ("%d", ptr ->data);  
        printf ("%d", ptr ->data);  
        return;  
    }  
}
```

$T(n)$ No. of values printed

$$T(n) = 3T(n-1) + 3$$

$$T(1) = 3$$

$$T(2) = 3T(1) + 3$$

$$= 3 \times 3 + 3 = 12$$

$$T(3) = 3T(2) + 3$$

$$3 \times 12 + 3 = 39$$

Slide The number of values printed by the program

Question



Consider the following function on linked list 12->45-30->44->45->45 and num as 45

```
2 3 int Dosomething(struct node* head, int num) {  
    struct node* current = head;  
    int count = 0; ✓  
    while (current != NULL) {  
        if (current->data == num)  
            count++;  
        current = current->next;  
    }  
    return count;  
}
```

} The return value of the function is

(A) 1

(B) 2

✓ (C) 3

(D) 4

Current → next != Null

Stops at Last Node

frequency of an element
count

Question

What is output of the program if the list contains 1 → 2 → 3?

HP is global variable

```
void Dosomething(){
```

```
    node *q, *r, *s;
```

```
    q = HP;
```

```
    r = NULL;
```

```
    while( q!=NULL){
```

```
        s = r;
```

```
        r = q;
```

```
        q = q → link;
```

```
        r → link = s;
```

```
    }
    HP = r;
```

```
}
```

HP: 300

A) 1 → 2 → 3 → NULL

☒ B) 3 → 2 → 1 → NULL

C) 2 → 1 → 3 → NULL

D) 3 → 1 → 2 → NULL

Reversal of
Single linked
List



100



200



300

q: 100

r: 100

s: 100

q: 200

r: 200

s: 200

q: 300

r: 300

q: NULL

Topic Question

The following C function takes a single-linked list of integers as a parameter and rearranges the element of the list. The function is called with the list containing the integers 1, 2, 3, 4, 5, 6, 7 in the given order. What will be the contents of the list after the function completes execution?

```
struct node {
    int value;
    struct node *next;
};
```


Topic Question

```
void rearrange (struct node *list) {
    struct node *p, *q;
    int temp;
    if (!list | | !(list->next)) return;
    p = list; q = list -> next;
    while (q) {
        temp = p ->value; p -> value = q -> value;
        q -> value = temp; p = q ->next;
        if(p) q = p->next; else q=0;
    }
}
```

0-Node

1-Node

2 1 4 3

1 → 2 → 3 → 4 → 5 → 6 → 7

p q p q

(A) 1, 2, 3, 4, 5, 6, 7

✓ (B) 2, 1, 4, 3, 6, 5, 7

(C) 1, 3, 2, 5, 4, 7, 6

(D) 2, 3, 4, 5, 6, 7, 1

[B]

Question



```
void foo(node *s) {  
    node *p=s;  
    if(s){  
        if(s->next == NULL) {  
            free(s);  
            HP = NULL;  
        }  
  
        else{  
            p=s->next;  
            free(s);  
            s= NULL;  
            HP = p;  
        }  
    }  
}
```

If the list contains 60-> 42->26->12->NULL

What will be the content of the list if we call

foo(HP)

A. 60 -> 13 -> 42 -> 26 -> 12-> NULL

B. 60 -> 42 -> 26 -> 12 -> NULL

C. 13 -> 60 -> 42 -> 26 -> 12 -> NULL

✓ D. 42 -> 26 -> 12 -> NULL

Question

Delete the first Node



```
void foo(node *s){
```

```
    node *p=s;
```

```
    if(s){
```

```
        if(s->next == NULL){
```

```
            free(s);
```

```
            HP = NULL;
```

```
        }
```

```
    else{
```

```
        p=s->next;
```

```
        free(s);
```

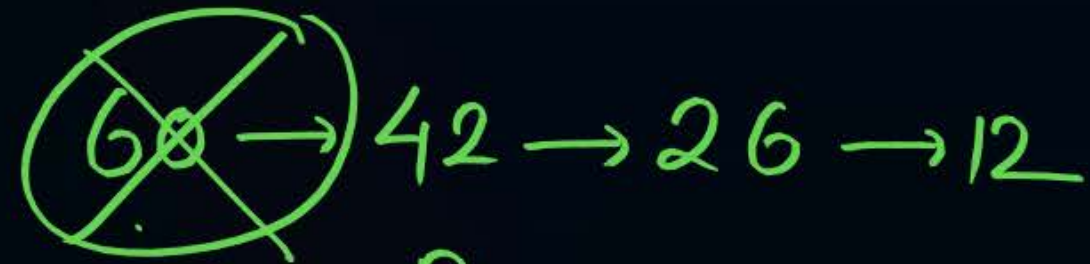
```
        s= NULL;
```

```
        HP =  p;
```

```
    }
```

```
}
```

```
}
```



P

HP = p Head pointer is p



↑
free(s)

HP = NULL

1 Node

Delete 1st
Node of list

Question

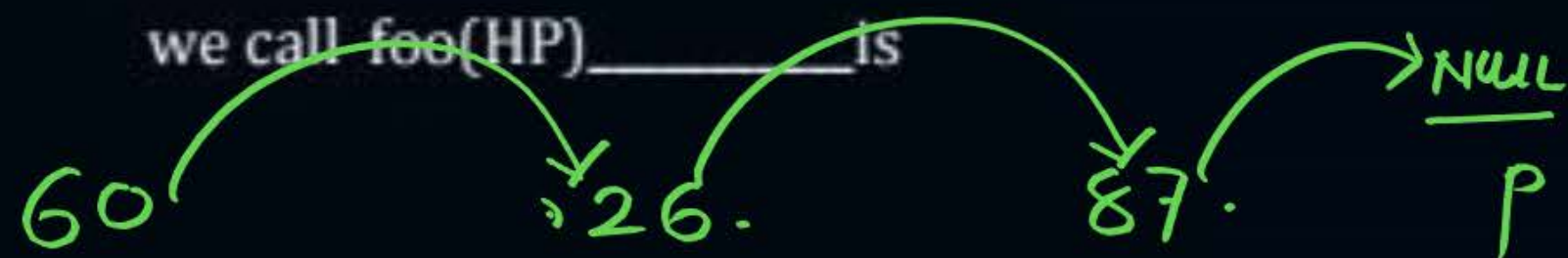


```
void foo(node* head) {  
    if (head == NULL) return;  
  
    struct node* prev = head;  
    struct node* curr = head->next;  
  
    while (prev != NULL && curr != NULL) {  
        prev->next = curr->next; ✓  
        free(curr); ✓  
  
        prev = prev->next;  
        if (prev != NULL)  
            curr = prev->next;  
    }  
}
```

If the list contains 60 -> 42 -> 26 -> 12 -> 87

98 -> NULL. The number of nodes in the list if

we call foo(HP) _____ is





Topic : Single Linked List

If the list contains 12->45-30->44->45->45 then value return by the following function if address of head node is passed is 44

```
int bar(node *q){
```

```
node *s=q, *f=q ;
```

```
while(f != NULL && f->next != NULL){
```

```
    s = s->next;
```

```
    f = f->next->next;
```

```
    }
```

```
    return s->data;
```

```
}
```

12 → 45 → 30 → (44) → 45 → 45

s s f

f s f

s

f = NULL

Middle element of List



Topic : Single Linked List

If the list contains 12->45-30->44->45->45->99 then value return by the following function if address of head node is passed is

```
int bar(node *q) {  
    node *s=q,*f=q ;  
    while(f != NULL && f->next != NULL){  
        s= s->next;  
        f = f->next->next;  
    }  
    return s->data;  
}
```

Advantage of linked list

- Linked List is *Dynamic data Structure*.
- Linked List *can grow and shrink during run time*.
- *Insertion and Deletion* Operations are Easier *Shifting Not Needed*
- *Efficient Memory Utilization*, i.e. no need to pre-allocate memory
- Faster Access time, can be expanded in *constant time without memory overhead*
- Linear Data Structures such as Stack, Queue can be *easily implemented* using Linked list

Disadvantage of the Single linked list

- Need to maintain *extra information* per node
- No *random access*
- Heap space restriction, *memory management is* user responsibility
- *Backward traversing* is not possible

Doubly Linked List Node



```
typedef struct dnode{
    int data;
    struct dnode * prev;
    struct dnode * next;
} dNode;
```




Getdnode and Build123 function

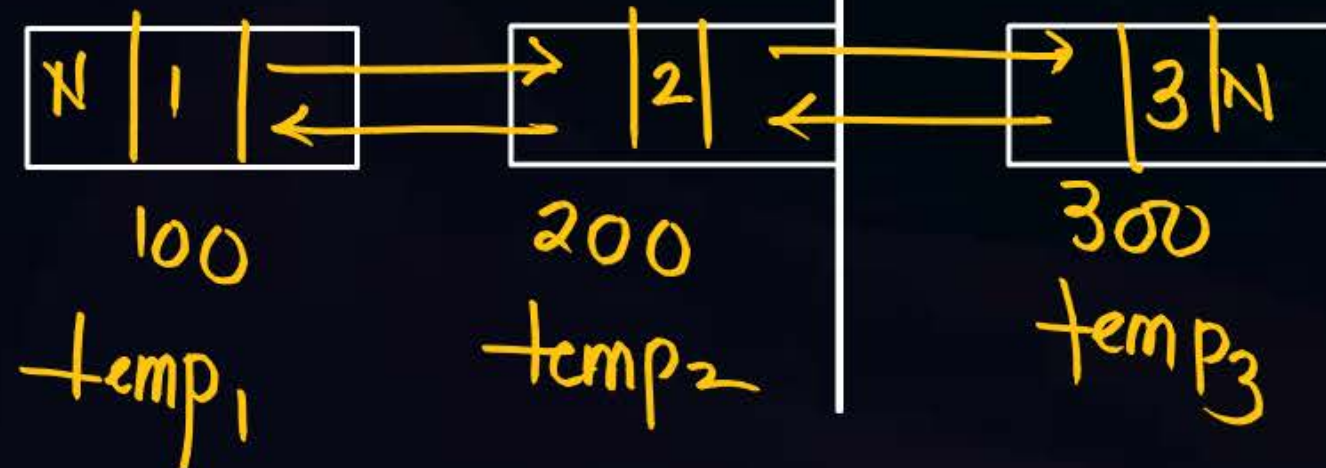
```
dNode* getdnode(int x){  
    dNode* temp = malloc(sizeof(dNode));  
    temp->data = x;  
    temp->prev = NULL;  
    temp->next = NULL;  
    return temp;  
}
```

Null	x	Null
------	---	------



Topic : Doubly Linked List

```
dNode* getdnode(int x) {  
    dNode *p;  
    p = malloc(sizeof(dNode));  
    p->data = x;  
    p->next = NULL;  
    p->prev = NULL;  
    return p;  
}
```



```
dNode* Build123() {  
    dNode *temp, *temp2, *temp3;  
    temp1 = getdnode(1);  
    temp2 = getdnode(2);  
    temp3 = getdnode(3);  
    temp1->next = temp2;  
    temp2->next = temp3;  
    temp2->prev = temp1;  
    temp3->prev = temp2;  
    return temp1;  
}
```

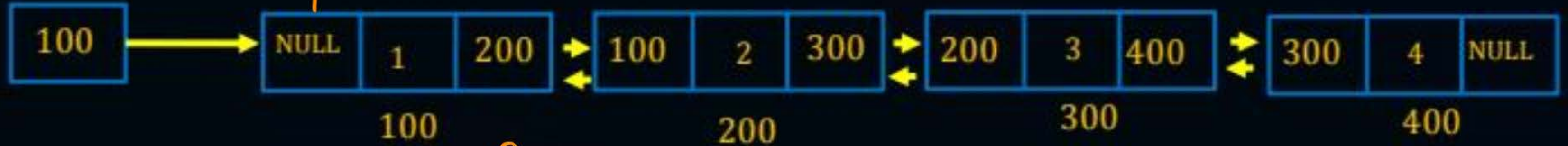



Topic : Doubly Linked List

Head pointer

$q = \text{NULL}$

$\text{Node } *p, *q;$



$\text{for (int } i = 1; i \leq 3; i++) \{$

One problem

HP is not updated

$\text{if } (i == 1)$

$\text{HP} = p$

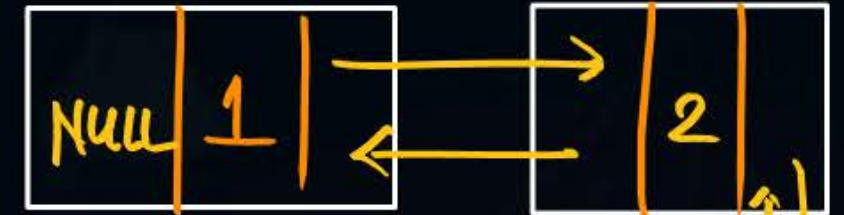
$p = \text{getnode}(i);$

$\text{if } (q) \quad q \rightarrow \text{next} = p;$

$p \rightarrow \text{prev} = q;$

$q = p;$

}



~~q~~

Question

```
void foo(dNode *s, int x){
    dNode *p = getdnode(x);
    if(s==NULL)
        HP = p;
    else{
        p->next=s;
        s->prev=p;
        HP = p;
    }
}
```

If the list contains 60 $\leftrightarrow$ 42 $\leftrightarrow$ 26 $\leftrightarrow$ 12 $\leftrightarrow$ NULL

What will be the content of the list if we call

foo(HP, 70)

A. 60 -> 13 -> 42 -> 26 -> 12 -> NULL

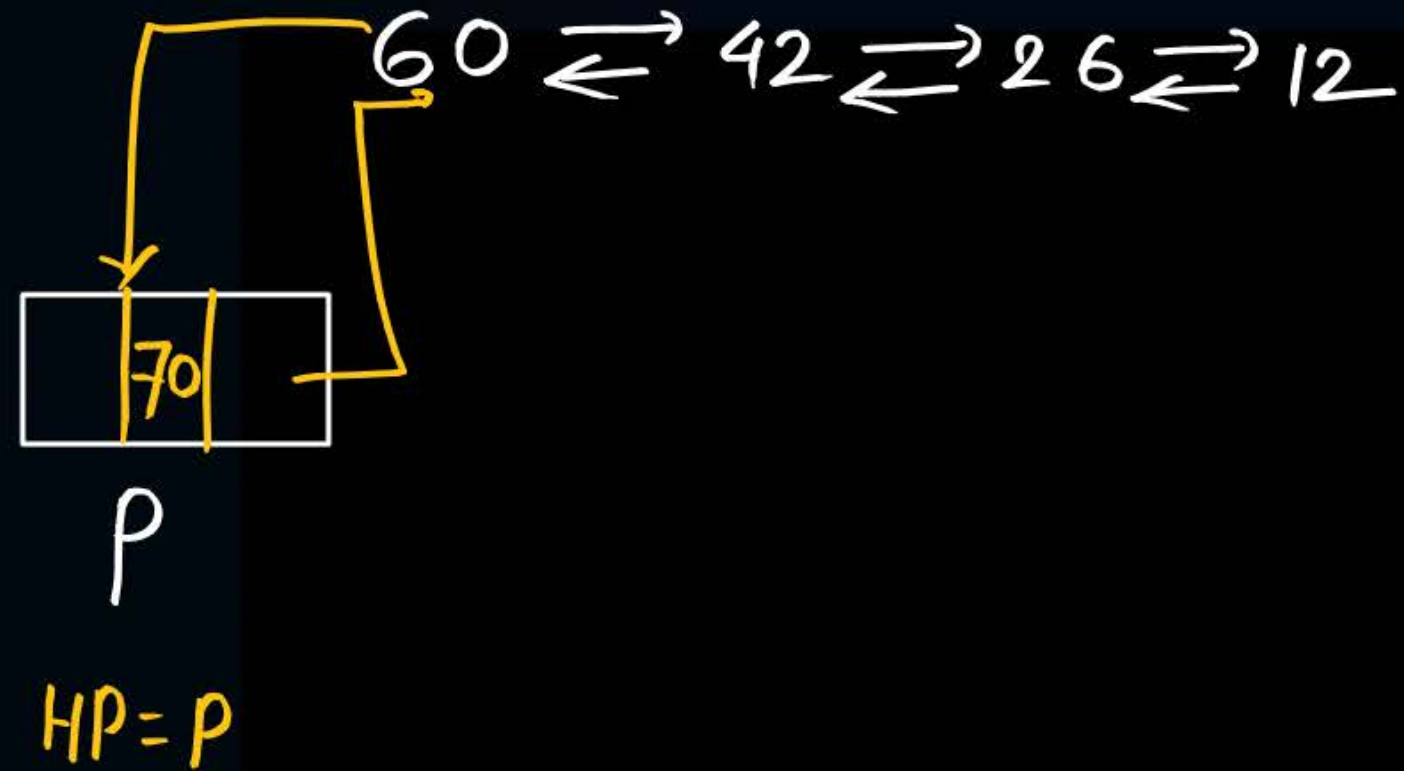
☒ B. 70->60 -> 42 -> 26 -> 12 -> NULL

C. 13 -> 60 -> 42 -> 26 -> 12 -> NULL

D. 42 -> 26 -> 12 -> NULL

Question

```
void foo(dNode *s, int x){
    dNode *p = getnode(x);
    if(s==NULL)
        HP = p;
    else{
        p->next=s;
        s->prev=p;
        HP = p;
    }
}
```



Adding a node at beginning of doubly linked list

Question

```
void addxaftery(dNode*s, int x, int y){
    dNode*p = getnode(x);
    if(s==NULL)
        return;
    else {
        while(s->next!=NULL && s->data!=y)
            s = s->next;
        if(s->data== y){
            p->next = s->next;
            s->next->prev = p;
            s->next = p;
            p->prev = s;
            return;
        }
    }
}
```

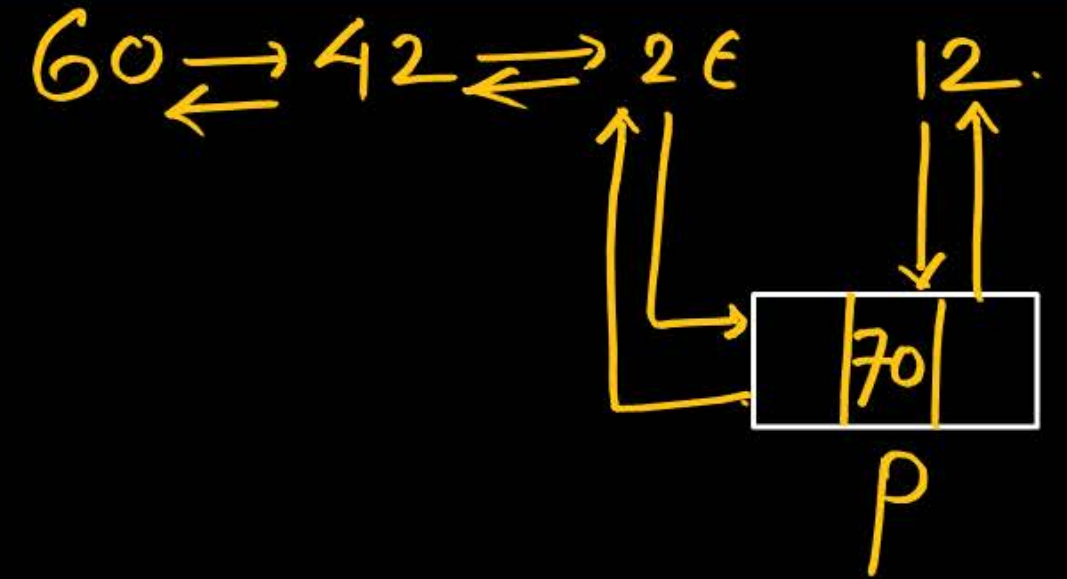
If the list contains 60-> 42->26->12->NULL
What will be the content of the list if we call

foo(HP, 70, 26)

- A. 60 -> 13 -> 42 -> 26 -> 12-> NULL
- B. 70->60 -> 42 -> 26 -> 12 -> NULL
- C. 60 -> 42 -> 26 -> 70-> 12 -> NULL ✓
- D. 42 -> 26 -> 12 -> NULL

Question

```
void addxaftery(dNode*s, int x, int y){
    dNode*p = getnode(x);
    if (s==NULL)
        free(p);
        return;
    else {
        while (s->next!=NULL && s->data!=y)
            s = s->next;
        if (s->data== y) {
            p->next = s->next;
            s->next->prev = p;
            s->next = p;
            p->prev = s;
            return;
        }
    }
}
```



60, 42, 26, 70, 12

Question

```
void addxaftery(dNode*s, int x, int y){
```

```
    dNode*p = getnode(x);
```

```
    if(s==NULL)
```

```
        return;
```

```
    else {
```

*S → next → prev
NULL*

```
        while(s->next!=NULL && s->data!=y)
```

```
            s = s->next;
```

```
        if(s->data== y){
```

*NULL pointer
dereference*

```
            p->next = s->next;
```

```
            s->next->prev = p; X
```

```
            s->next = p;
```

```
            p->prev = s;
```

```
            return;
```

```
        } } }
```

If the list contains 60->42->26->12->NULL

What will be the content of the list if we call

foo(HP, 70, 12)

A. 60 -> 13 -> 42 -> 26 -> 12-> NULL

B. 70->60 -> 42 -> 26 -> 12 -> NULL

C. 60 -> 42 -> 26 -> 12->70->NULL

☒ D. Null Pointer dereferencing

60 ↔ 42 ↔ 26 ↔ ^S12

70	NULL
----	------

How to fix



```
void addxaftery(dNode*s, int x, int y){
    dNode*p = getnode(x);
    if(s==NULL)
        return;
    else {
        while(s->next!=NULL && s->data!=y)
            s = s->next;
        if(s->data== y){
            p->next = s->next;
            if(s->next) s->next->prev = p;
            s->next = p;
            p->prev = s;
            return;
        }
    }
}
```

Consider the following ANSI C program:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{
    int value;
    struct Node *next;};
```




Consider the following ANSI C program:

```
int main () {  
    Struct Node *boxE, *head, *boxN; int index = 0;  
    boxE = head = (struct Node*) malloc (sizeof (struct Node));  
    head->value = index;
```

```
    for (index = 1; index <=3; index++) {
```

```
        box N = (struct Node *) malloc (sizeof (struct Node));
```

```
        BoxE->next = boxN;
```

```
        boxN->value = index;
```

```
        boxE= boxN; }
```

```
    for (index = 0; index <=3; index++) {
```

```
        printf( Value at index %d is %d\n , index, head->value);
```

```
        head = head->next;
```

```
        printf ( Value at index %d is %d\n , index + 1, head->value);
```

```
    }
```

index=0

0, 0, head

1 1

=200

index=1

1, 1 head:300

2, 2

index=2

2, 2 head:400

3 3

index=3

3, 3

4

head = garbage

head
100

0 |
100

1 |
200

2 |
300

3 |
400

Not NULL

boxE: ~~100~~ ~~200~~ ~~300~~ 400



Consider the following ANSI C program:

```
int main () {
    Struct Node *boxE, *head, *boxN; int index = 0;
    boxE = head = (struct Node*) malloc (sizeof (struct Node));
    head->value = index;
    for (index = 1; index <=3; index++) {
        box N = (struct Node *) malloc (sizeof (struct Node));
        BoxE->next = boxN;
        boxN->value = index;
        boxE= boxN; }
    for (index = 0; index <=3; index++) {
        printf( Value at index %d is %d\n , index, head->value);
        head = head->next;
        printf ( Value at index %d is %d\n , index + 1, head- >value);
    }
}
```


Which one of the statements below is correct about the program?

- (A) It dereferences an uninitialized pointer that may result in a run-time error ✓
- (B) It has a missing return which will reported as an error by the compiler ✗
- (C) Upon execution, the program creates a linked-list of five nodes ✗
- (D) Upon execution, the program goes into an infinite loop ✗

Last Lectures of
Linked List

THANK - YOU